LAPORAN PRAKTIKUM 4 BIG DATA
NoSQL dan Pemodelan Data



Disusun Oleh:

Stanley Leonard Darmali/Kelas B

2311532021

Dosen Pengampu:

Afdhal Dinilhak

Ajeng Dwi Asti

Departemen Informatika

Fakultas Teknologi Informasi

Universitas Andalas

2026

Repository Github:
https://github.com/AbyssWatcher07/BigData26_B_2311532021_StanleyLeonardDarmali

Sumber Data & Etika Penggunaan Dataset

- TLC Yellow Taxi & Taxi Zone (Januari & Juli 2023): Data transaksi nyata taksi kuning New York yang diunduh dari repositori publik resmi NYC TLC dan dilanjutkan pengelolaannya dari artefak Praktikum 3. Seluruh atribut bersifat open data dan telah di-anonimisasi (bebas dari PII).

- Sintetis Spasial: Titik koordinat (lintang-bujur) yang digunakan pada Bagian K-6 merupakan data sintetis yang dibangkitkan secara prosedural. Hal ini dilakukan karena dataset otentik NYC TLC modern tidak lagi memuat data koordinat geospasial.

- Etika Kinerja: Seluruh pengujian benchmarking (waktu eksekusi memori) dijalankan secara lokal di Colab menggunakan fungsi ukur(), tanpa membanjiri API atau server eksternal, menjaga etika konsumsi bandwidth.

1. Langkah-langkah Kerja (K-1 s.d. K-9)

- K-1. Menyiapkan Lingkungan dan Data Contoh: Menyiapkan dependensi multi-database (DuckDB, NetworkX, TinyDB, DeltaLake) dan memuat 2.986.910 baris data Parquet yang tersisa dari pembersihan Praktikum 3 tanpa menyatukan null dictionaries yang tidak kompatibel di PyArrow.

- K-2. Tabel Biasa vs Key-Value: Membandingkan pencarian data menggunakan SQLite (dengan dan tanpa indeks) dan Dictionary JSON. Pencarian dengan kunci (indeks) terbukti sangat cepat, namun Dictionary sangat lambat saat menghitung nilai agregasi ($O(N)$ iterasi JSON).

- K-3. Document: Bentuk Data yang Fleksibel: Mengimpor data ke dalam dokumen bersarang TinyDB. Diuji untuk menyimpan kolom tidak beraturan (seperti `tip` yang hanya ada pada kartu kredit atau `cuaca` yang disuntikkan secara dinamis), membuktikan sifat schema-less NoSQL.

- K-4. Penyimpanan per Kolom (DuckDB pada Parquet): Menjalankan eksekusi analitik langsung di atas folder Parquet terpartisi tanpa memuatnya ke memori. DuckDB berhasil membaca partisi hive Staten Island secara kilat melalui mekanisme partition pruning.

- K-5. Graph: Pertanyaan tentang Keterhubungan: Mengubah data zona menjadi Nodes dan frekuensi perjalanan menjadi Edges menggunakan NetworkX. Melakukan penjelajahan rute 2 langkah (Depth traversals) yang terbukti jauh lebih ringkas dari sintaks self-join SQL bertingkat.

- K-6. Indeks Spasial: Mensimulasikan titik koordinat taksi dan membuat pengujian 1.000 wilayah kotak bounding box. Penggunaan struktur pohon STRtree mempercepat penapisan area dibanding teknik broadcasting array matriks NumPy murni.

- K-7. Delta Lake: Versi, Time Travel, dan Schema Enforcement: Mengkonversi data ke format DeltaLake dan mendemonstrasikan konsistensi transaksional ACID. Perubahan tarif yang di-

update disimpan sebagai versi baru, sementara skema ilegal langsung ditolak tanpa merusak data dasar.

- K-8. Small File Problem pada Tabel Delta: Mensimulasikan kedatangan 20 batch data kecil dan memadukannya melalui fungsi `dt.optimize.compact()`. Jumlah file aktif ditekan kembali menjadi 1, namun log transaksi sebelumnya ditahan di disk.

- K-9. Merangkum dan Memilih: Mengekspor seluruh durasi tolok ukur eksekusi setiap metode penyimpanan dan mengumpulkannya ke file `benchmark_nosql.csv` lokal.

2. Analisis Hasil (O.1 s.d. O.7)

O.1 Pencarian satu data. Berapa kali lebih lambat tabel tanpa indeks dibanding tabel berindeks? Berapa kali lebih lambat tabel berindeks dibanding key-value? Pada selisih mana perbedaan itu benar-benar berarti?

Tabel tanpa indeks (6,6082 detik) memakan waktu sekitar ~1.573 kali lebih lambat dibandingkan tabel dengan indeks (0,0042 detik). Tabel berindeks sedikit lebih lambat (~3,2 kali) dibandingkan key-value (0,0013 detik). Perbedaan yang benar-benar bermakna secara arsitektural terjadi pada selisih antara pencarian berindeks dan tanpa indeks, karena perbandingan 6 detik dan hitungan mikrodetik akan menentukan kelumpuhan sistem (bottleneck) di lingkungan produksi berskala masif.

O.2 Harga kesederhanaan. Berapa kali lebih lambat key-value untuk menghitung dibanding SQL GROUP BY, dan mengapa?

Untuk tugas menghitung/agregasi wilayah, key-value (1,3900 detik) berjalan hampir 7 kali lebih lambat dibanding SQL GROUP BY (0,1998 detik). Alasannya adalah karena penyimpanan key-value meratakan data sebagai teks, sehingga program harus memanggil `json.loads` untuk mengekstrak, mengonversi teks ke numerik, lalu menghitungnya baris demi baris, sedangkan memori SQL terstruktur membaca kolom memori secara spesifik.

O.3 Data fleksibel. Sebutkan satu keuntungan dan satu risiko dari kenyataan bahwa dokumen ke-20.001 boleh membawa field baru.

Keuntungannya adalah aplikasi dapat memasukkan rekaman baru yang bentuknya berevolusi (seperti menambahkan blok 'cuaca') kapan pun secara instan (fleksibilitas eksekusi parsial). Namun, risikonya adalah tidak ada Schema Enforcement dari database, sehingga kesalahan pengetikan format atau nama kunci akan lolos sebagai polusi metadata yang merusak skrip agregasi akhir.

O.4 Penyimpanan per kolom. Berapa kali lebih cepat DuckDB dibanding SQLite pada data Anda? Sebutkan dua penyebabnya, dan satu di antaranya yang bukan soal format kolom.

DuckDB (0,3062 detik) sekitar 10 kali lebih cepat memproses agregasi wilayah dibandingkan SQLite (3,0897 detik). Dua alasan utamanya adalah: (1) DuckDB menggunakan arsitektur

pemrosesan data vektoral antar core CPU yang mengandalkan komputasi paralel (ini BUKAN soal format kolom, melainkan processing engine), dan (2) fenomena Column Pruning, yang murni karena format Parquet, di mana mesin hanya memuat kolom spesifik dari disk.

O.5 Graph. Bila dua langkah pada graph dan SQL sama cepatnya, apa yang membuat graph tetap layak dipilih untuk pertanyaan keterhubungan? Rujuk hasil untuk k=1 sampai 4.

Meskipun waktu eksekusi untuk 2 langkah cukup mirip (Graph 0,0062 detik vs SQL 0,0112 detik), rancangan Graph menang mutlak dalam fleksibilitas pertanyaan. Kita cukup menulis `cutoff=k` (k=1 s.d. 4) di NetworkX untuk menambah tingkat penelusuran. Jika memakai SQL, kita akan dipaksa menulis satu kueri Self Join masif bertingkat setiap kali jumlah lompatan 'k' direvisi.

O.6 Indeks spasial. Berapa kali lebih cepat STRtree dibanding loop Python, dan dibanding NumPy? Mengapa dua angka itu berbeda jauh, dan apa artinya bagi cara Anda menulis klaim percepatan?

STRtree (0,3236 detik) lebih cepat dari iterasi NumPy (0,5383 detik), dan ekstremnya lebih cepat ratusan kali lipat dari iterasi Loop tunggal. Perbedaan jauh ini terjadi karena NumPy beroperasi secara vectorized di memori RAM, sementara Python Loop menghitung secara sekuensial baris per baris. Artinya, klaim 'indeks spasial 1000x lebih cepat' sangat menyesatkan jika pembandingnya salah sasaran.

O.7 Delta Lake. Tunjukkan dari keluaran K-7 bahwa (a) update tidak menghilangkan data lama, dan (b) penulisan yang gagal tidak meninggalkan jejak. Jelaskan juga mengapa setelah compaction file di disk masih lebih banyak daripada file aktif.

(a) Update 20 baris di Bronx menggeser history pointer ke versi 2, namun data versi 0 (100.000 baris) dan versi 1 (150.000 baris) masih dapat di-query dan dikembalikan secara sempurna. (b) Penulisan `kolom_asing` secara gagal tidak meninggalkan residu korup, dan log transaksinya tetap mandek di versi 2. File disk menumpuk (24) lebih banyak dari active file (1) setelah kompaksi karena sistem menahan file lama (`00.json`, `01.json`) sebagai bantalan undo sebelum `VACUUM` fisik membersihkannya.

3. Studi Kasus Perancangan Penyimpanan (Bagian P)

Kasus: rancangan penyimpanan untuk aplikasi taksi.
Sebuah aplikasi taksi ingin membuat lima fitur di atas data perjalanan yang sama:

1. Halaman 'detail perjalanan' yang dibuka penumpang dan harus terbuka dalam hitungan milidetik, berdasarkan id perjalanan.

2. Dasboard pendapatan per wilayah per jam untuk manajemen, dengan filter tanggal dan wilayah.

3. Fitur 'zona yang sering dituju dari sini' dan 'zona yang bisa dicapai dalam dua atau tiga langkah' untuk rekomendasi rute.

4. Pencarian 'taksi yang pernah menjemput dalam radius 500 meter dari titik ini'.

5. Audit keuangan: tim keuangan berhak meminta 'kembalikan tabel tarif ke keadaan tanggal 1' bila ada koreksi yang keliru.

Fitur	Cara Menyimpan	Alasan	Kelemahan
Detail perjalanan (ID)	Key-Value / SQL Index	Pencarian ID instan (0.0013s) karena akses memori langsung O(1).	Terkunci pada ID, lambat di-agregasi.
Dashboard manajemen	DuckDB (Parquet)	Format kolom & Partition Pruning memangkas I/O (cepat: 0.3062s).	Lambat mengubah nilai baris tunggal.
Rekomendasi Rute	Graph (NetworkX)	Penelusuran dinamis O(k) (0.0062s) tanpa Self-Join berjenjang.	Overhead memori RAM yang tinggi.
Radius Taksi	Spatial (STRtree)	Memfilter dan memangkas pencarian poligon secara logaritmik (0.32s).	Lemah pada bounding box ukuran makro.
Audit Keuangan	Delta Lake	Menjamin jejak ACID & time-travel restorasi versi absolut.	Rentan small-file problem tanpa kompaksi.

Kondisi Redundansi: Dataset terpaksa direplikasi (contoh: salinan logis statis di Graph untuk algoritma rute, dan arsip data utamanya di Delta Lake). Kedua salinan dijaga agar sinkron menggunakan mekanisme Change Data Capture (CDC) inkremental pada jam kosong, menjadikan Delta Lake sebagai Single Source of Truth.

4. Hasil Latihan Mandiri (Bagian Q)

- Q.1 Latihan 1 (Pencarian 2000 id): Eksekusi mencari 2000 id memecah batasan dengan pencatatan SQL Index berjalan di kisaran rata-rata 18.8 mikrodetik per pencarian, sedikit di bawah Key-Value yang melaju di 6.6 mikrodetik.

- Q.2 Latihan 2 (Dokumen vs SQL): Kueri TinyDB dan SQL tidak menghasilkan nilai sama (TinyDB = 0, SQL = 3) karena struktur bersarang (nested key) `metode_bayar` pada document store belum ditulis seragam di kueri eksperimen Python (inkonsistensi data-type mismatch).

- Q.3 Latihan 3 (DuckDB SUM): Eksekusi agregasi DuckDB membengkak dari 0.1999 detik (1 kolom) menjadi 0.3597 detik (4 kolom). Hal ini membuktikan utilitas krusial Column Pruning, di mana pembacaan spesifik selalu menghemat IO disk.

- Q.4 Latihan 4 (Out-degree Graph): Hasil kalkulasi Out-degree jaringan Graph 100% kongruen dengan kueri relasional `COUNT(DISTINCT tujuan)` di SQL (Zona 132 memimpin dengan 260 rute distingtif).

- Q.5 Latihan 5 (Skala Spasial STRtree vs Numpy): Pada bounding box makro (0.02 derajat), NumPy (0.52s) lebih tangguh dari STRtree (3.03s) karena indeks STRtree terlalu banyak me-load tumpang tindih area (overlap). Namun di kotak mikro presisi (0.001 derajat), STRtree mendominasi seketika (0.02s) karena mampu memangkas penjelajahan daun secara absolut.

- Q.6 Latihan 6 (Restore vs Time Travel): Time travel sekadar 'melihat' masa lalu melalui kacamata sementara, sedangkan fungsi `dt.restore(1)` mensimulasikan mesin waktu fisik yang memaksa kondisi tabel 160k kembali menjadi 150k. Restore ini dicatat sebagai Versi 24 (versi baru) agar sejarah transaksi tetap berjalan maju secara linier.

5. Tugas Praktikum (Bagian R)

R.1 Rekap Benchmark Nosql

Kinerja waktu memori (benchmark) telah diakumulasikan sepanjang praktikum. Relasional (muat tabel 2.26s) dan Key-Value (bangun dict 9.41s) mendemonstrasikan bahwa proses inisialisasi awal (hydration) KV jauh lebih lambat dari SQL, namun pencarian id spesifik KV (0.0013s) merupakan opsi absolut tercepat yang tersedia.

R.2 Data lain, cara lain. Pilih satu data lain dan rancang tiga cara menyimpannya (tabel, document, dan graph atau key-value). Untuk masing-masing, tulis satu pertanyaan yang mudah dan satu yang canggung dijawab.

Data yang dipilih: Entitas spasial 'Zona Jemput'. Data direplikasi ke dalam tiga ekosistem penyimpanan:

- Relasional (SQLite):
  - Mudah: Menjawab pertanyaan 'Berapa banyak zona di wilayah Queens?' dengan agregasi relasional logis (68 zona).
  - Canggung: Menyimpan koordinat spasial batas poligon tiap zona yang ukurannya dinamis (skema relasional kaku terhadap array spasial).

- Document (TinyDB):
  - Mudah: Mendokumentasikan properti asimetris secara dinamis (misal: EWR memiliki 1 field

terminal asimetris yang zona lain tidak punya).
  - Canggung: Melakukan operasi JOIN agregasi raksasa kembali ke tabel transaksi taksi.

- Key-Value (Dictionary):
  - Mudah: Melakukan lookup nama wilayah seketika berbekal kuncinya secara O(1) (Zona:132 -> Queens).
  - Canggung: Kueri 'Sebutkan semua ID zona di wilayah Queens!' (Karena mengharuskan full table scan membuka isi seluruh value di dalam database).

R.3 Delta Lake dua hari. Simulasikan dua hari kerja: hari pertama menulis sebagian data, hari kedua menambah data dan mengoreksi beberapa baris.

Simulasi penyimpanan terpusat Lakehouse mendemonstrasikan transisi ekosistem 2 hari. Sistem dipasok 10.000 baris awal di hari ke-1, lalu diinjeksi (append) dan direvisi tarif logisnya di hari ke-2, mencapai batas 20.000 entitas. Fitur log transaksi menghasilkan `00.json`, `01.json`, dan `02.json` yang melacak revisi tersebut dan memungkinkan pembuktian pembacaan utuh time-travel atas memori 10.000 baris di akhir hari ke-1.

6. Pertanyaan Evaluasi (Bagian S)

S.1 Mengapa NoSQL sering dibaca "Not Only SQL"? Sebutkan dua situasi ketika tabel relasional tetap pilihan yang lebih baik.

NoSQL tidak mematikan SQL, melainkan memberinya kawan fungsional. Tabel relasional murni tetap mutlak unggul jika: (a) Transaksi menuntut tingkat keakuratan ACID mutlak (misal: mesin deposit perbankan), dan (b) Format entitas data sangat statis, tidak berubah, dan sangat padat koneksi JOIN multi-tabel.

S.2 Apa perbedaan BASE dan ACID? Jelaskan teorema CAP dengan satu contoh data pada aplikasi taksi yang sebaiknya memilih consistency, dan satu yang boleh memilih availability.

ACID (Relasional) memprioritaskan kekokohan transaksional di mana jika satu tahap batal, seluruh write dibatalkan. BASE (NoSQL) mengorbankan kepastian tersebut (Eventual Consistency) agar database tidak pernah putus koneksi. Berdasarkan Teorema CAP: Sistem aplikasi taksi wajib memilih Consistency pada proses pemotongan saldo e-wallet pengguna, namun sangat dianjurkan memilih Availability saat menampilkan daftar riwayat titik jemput terdekat di HP supir.

S.3 Mengapa pencarian satu data pada key-value sangat cepat, tetapi menghitung sangat lambat? Apa yang dikorbankan key-value demi kesederhanaannya?

Kinerja KV mendominasi di lookup titik ID O(1) karena kunci hash-nya merujuk langsung memori fisik. Namun performanya musnah saat diagregasi, karena nilai datanya dikunci (encrypted) sebagai blok teks string. Key-Value mengorbankan interaktivitas kolom demi kesederhanaan scaling-out server terdistribusi.

S.4 Jelaskan perbedaan menyatukan data (embedding) dan merujuk (referencing) pada model document, dan satu kondisi yang membuat masing-masing lebih tepat.

Embedding (menyatukan bersarang): Sangat ideal untuk kueri terisolir (Read-Heavy) yang isinya tidak akan direvisi lagi (seperti mencetak lembar kuitansi/faktur riwayat taksi yang lampau). Referencing (mengaitkan ID ke tabel lain): Dipakai mutlak jika data induk selalu berubah dan ditonton miliaran user serentak (seperti ID plat armada mobil).

S.5 Kapan graph lebih unggul dari SQL untuk pertanyaan keterhubungan? Berikan alasan yang tidak bergantung pada kecepatan.

Graph jauh lebih unggul dalam pertanyaan berbasis logika lintas arah bersarang (Variable Depth atau Shortest Path). Dengan Graph, pemrogram Python hanya perlu mengubah angka parameter (misal $K=2$ jadi $K=6$), sedangkan dengan SQL, satu modifikasi lompatan mengharuskan penulisan ulang kueri sub-tabel tipe SELF JOIN secara eksplisit, panjang, dan rawan error.

S.6 Apa yang dilakukan indeks spasial sebelum memeriksa lokasi secara tepat? Mengapa klaim "indeks 1.000 kali lebih cepat" bisa menyesatkan?

Indeks spasial (STRtree) terlebih dahulu membungkus poligon titik menggunakan matriks bujur sangkar hipotetis (Kotak Bounding Box) lalu mengeleminasi petak terluar secara kasar. Klaim '1.000x' manipulatif karena NumPy (tanpa STRtree) sebetulnya bisa melawan kecepatan tersebut bila rentang pencariannya (Derajat Radius) tidak merusak batasan overlap wilayah (seperti dibuktikan dalam Latihan Q.5).